All told, that's over 40 lines of code to test a 32-line view. That's kind of typical. Writing tests that check all the conditions of your code tends to take about the same number of lines of code as the code under test. If writing automated tests isn't something you do at work, changing to this habit can be startling to your schedule. Suddenly, you're writing half the number of features in the same time because half the time you're writing tests. This can make management cranky. Long term, though, this is a far more efficient way of coding. What you're really doing is reducing the future test time, time that wasn't originally accounted for in the development portion of your schedule. With tests in place, you can quickly test the entire system as frequently as you like, something you can't do if you stick with a manual testing approach.

8.6 Testing with file uploads

There is one more scenario that the test for `edit_venue()` still doesn't cover: associating an image with the venue. This makes life a little more complicated than just having some text values in the POST dictionary. There are a couple of ways to handle this. First, you could keep a file with your tests, read it, and upload its contents as part of your test. Alternatively, you can create the file on the fly.

Django provides the `SimpleUploadedFile` class as a way to build files. The class gets used if you're writing a custom file handler, but is equally useful when testing. `SimpleUploadedFile` takes two arguments when constructed: the name of the file and its binary contents. As the `Venue` object's associated file is a picture, the binary contents need to pass the is-an-image validation performed by the underlying `ImageField`. To save you some time (this is a full service book!) I created a 1-pixel GIF, JPG, and PNG and discovered that the GIF is the smallest. Using Python's `base64` library, I encoded the GIF into a string, which gets hardcoded in the test. When the `SimpleUploadedFile` gets created, the base64 string gets decoded into a binary representation of the GIF. I put this code in the `.setUp()` method, as it may be useful in another test (spoiler alert: it gets used in the excercises).

> **WARNING** `SimpleUploadedFile` gets accessed like a stream. If you are testing the same value more than once in a method, re-create it to reset its starting point to the beginning.

If you were actually doing an HTTP call in your test, you would have to treat the file data specially. This is the same as the need to specify `encytype="multipart/form-data"` in the `<form>` tag. The Django test utilities are smart, though, and take care of this situation for you. They automatically recognize file handles in the `.client.post()` dictionary and put them in the right part of the request.

One last tricky little thing: uploaded files go somewhere. You probably don't want your test file to end up in your uploads folder. If you're as anal- retentive as I am, you want to remove any files after the test. There is a better solution though: Django provides a decorator that modifies a configuration value for the duration of a test. The decorator is named `@override_settings`, and it takes one or more keyword arguments, corresponding to the name and value of the configuration value you wish to change. A